

Mini-Project: Socratic AI Tutor

Course: DAT566/DIT408 – Introduction to Data Science and AI

Topic: Retrieval-Augmented Generation for Educational Assistants

Task: RAG-based tutor with hybrid retrieval, grounding checks, and evaluation

Group 77

Team Members

Name:	Mina Tahmasebi Berjouei
Program:	Information and Communication Technology, MSc
Email:	Berjouei@chalmers.se
Hours:	20

Name:	Olga Ratuszniak
Program:	Data Science and AI, MSc
Email:	olgara@chalmers.se
Hours:	20

March 20, 2026

Contents

1	Introduction	2
2	Motivation and Objectives	2
3	Relation to Course Concepts	2
4	System Design and Methodology	3
4.1	Architecture Overview	3
4.2	Ingestion Pipeline	3
4.3	Retrieval and Answer Pipeline	3
4.4	Persistence	4
5	Implementation	4
5.1	Hybrid Reranker	4
5.2	Grounding Check	5
5.3	Supported File Formats	5
6	Evaluation Framework	6
6.1	Golden Dataset	6
6.2	Metrics	6
7	Discussion: Mimicking vs. Understanding	6
8	Known Limitations	7
9	Conclusion	7
	Appendix: Key Implementation Components	8
	Appendix: Full Implementation Components	10

1. Introduction

This report presents a mini-project that builds a Socratic AI Tutor, an educational virtual assistant based on Retrieval-Augmented Generation (RAG). The system allows users to upload study files, store them as vector embeddings in a persistent database, and ask grounded questions that are answered using evidence retrieved directly from those files.

The project relates to the course concept of **conversational AI and chatbots**, and extends it in three concrete ways: by grounding responses in user-provided documents, by implementing a hybrid retrieval and reranking strategy, and by providing a quantitative evaluation framework for both retrieval quality and answer quality.

2. Motivation and Objectives

Large language models (LLMs) are prone to hallucination: generating plausible but factually incorrect answers. In an educational context this is particularly harmful. RAG addresses this by constraining the model to answer only from retrieved evidence, making responses verifiable and grounded.

Main Objectives

- Provide accurate, evidence-grounded answers from user-provided study files.
- Maintain persistent memory across application restarts.
- Allow users to manage the knowledge base: save, delete, or reset files.
- Evaluate retrieval and generation quality with measurable metrics.

3. Relation to Course Concepts

The project directly extends the course topic of **retrieval-augmented generation and conversational agents**. Standard RAG pipelines retrieve documents and pass them to an LLM. This project extends that baseline in three ways:

1. **Hybrid reranking**: retrieved chunks are reranked using a combination of semantic rank (MMR) and lexical overlap score, improving retrieval precision.
2. **Grounding checks**: before calling the LLM, a grounding strength score is computed between the query and retrieved context. If the score is below a threshold, the system refuses to answer rather than hallucinate.
3. **Evaluation framework**: a golden dataset with Hit@k, MRR, Token F1, and Semantic Similarity metrics provides quantitative feedback on both retrieval and generation quality, going beyond simple manual inspection.

The project also touches on the course discussion of **mimicking vs. understanding**: the tutor retrieves and reranks text but does not truly comprehend it. This distinction is discussed further in Section 7.

4. System Design and Methodology

4.1. Architecture Overview

The application is structured as three UI panels built with Streamlit:

- **Left panel:** knowledge base management (upload, save, list, delete, reset).
- **Center panel:** chat interface for user interaction with the tutor.
- **Right panel:** evaluation results and assessment.

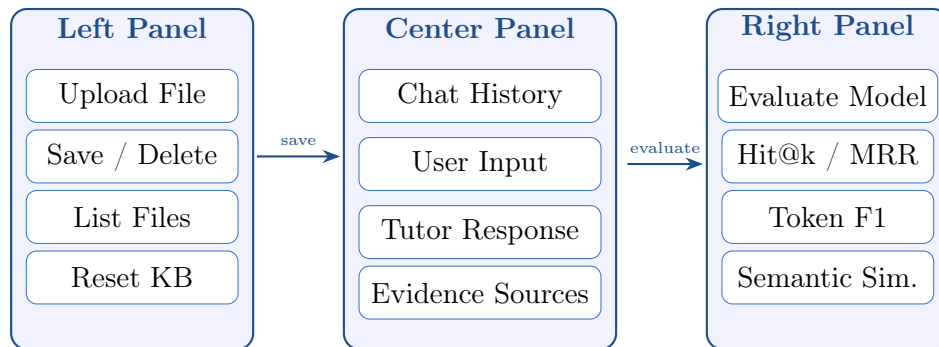


Figure 1: Three-panel Streamlit UI architecture.

File	Responsibility
<code>app.py</code>	Streamlit UI, session state, panel layout, button logic.
<code>database.py</code>	File parsing, chunking, embedding ingestion, hybrid retrieval, delete/reset.
<code>tutor_engine.py</code>	OpenAI calls, chitchat detection, grounding checks, fallback behavior.
<code>evaluation.py</code>	Golden dataset loading, Hit@k, MRR, Token F1, Semantic Similarity.
<code>config.py</code>	Runtime constants and environment variable mapping.
<code>docker-compose.yml</code> / <code>Dockerfile</code>	Containerized deployment with named volumes.

4.2. Ingestion Pipeline

1. User uploads a file (`.txt`, `.md`, `.pdf`, `.csv`, `.json`).
2. The file is parsed into clean plain text using format-specific readers.
3. Text is split into overlapping chunks using a recursive character splitter (chunk size: 450, overlap: 80).
4. Chunks are embedded with `sentence-transformers/all-MiniLM-L6-v2` and stored in ChromaDB with stable IDs to prevent duplicate insertion.

4.3. Retrieval and Answer Pipeline

1. User sends a message.

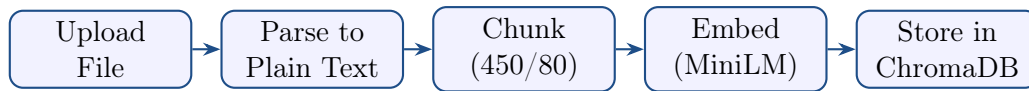


Figure 2: Document ingestion pipeline.

2. If the message is chitchat (greeting, thanks, goodbye), it is answered directly without retrieval.
3. Otherwise, top-k chunks are retrieved using MMR (Maximal Marginal Relevance) and reranked with a hybrid score: $0.7 \times \text{semantic rank} + 0.3 \times \text{lexical overlap}$.
4. A grounding strength score is computed. If below 0.18, the system declines to answer rather than risk hallucination.
5. An extractive answer is attempted first. If successful, it is returned directly. Otherwise, the full context and chat history are sent to the OpenAI API.
6. The answer and retrieved evidence metadata (source, page, chunk) are returned.

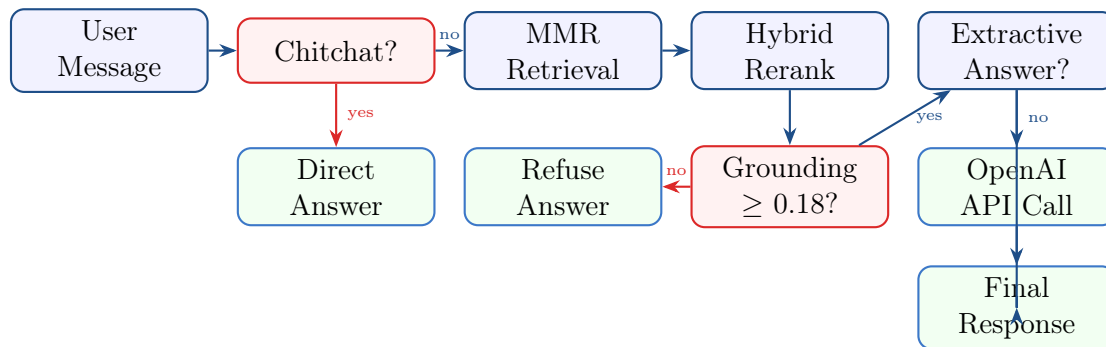


Figure 3: Retrieval and answer generation pipeline.

4.4. Persistence

Named Docker volumes (`aitutor_chroma_data`, `aitutor_uploads`) store the ChromaDB database and uploaded files independently of the source code. This ensures knowledge persists across container restarts and avoids accidental preloading from the local project folder.

5. Implementation

5.1. Hybrid Reranker

The reranker in `database.py` combines two signals: the inverse rank from MMR retrieval (semantic) and the token overlap ratio between the query and the chunk (lexical). After scoring, duplicate chunks sharing the same (`source`, `chunk_id`) pair are removed before returning the top-k results. This improves precision for keyword-heavy study material where pure semantic similarity may miss exact terms.

```

1 def _hybrid_rerank(self, query, docs, top_k):
2     q_tokens = self._tokenize(query)
  
```

```

3     scored = []
4     for rank, doc in enumerate(docs, start=1):
5         d_tokens = self._tokenize(doc.page_content)
6         lexical = len(q_tokens.intersection(d_tokens)) / max(1, len
            (q_tokens))
7         semantic = 1.0 / rank
8         score = 0.7 * semantic + 0.3 * lexical
9         scored.append((score, doc))
10    scored.sort(key=lambda x: x[0], reverse=True)
11    # Deduplicate by (source, chunk_id)
12    unique, seen = [], set()
13    for _, doc in scored:
14        key = (doc.metadata.get("source"), doc.metadata.get("
            chunk_id"))
15        if key not in seen:
16            seen.add(key)
17            unique.append(doc)
18        if len(unique) >= top_k:
19            break
20    return unique

```

Listing 1: Hybrid reranking score in database.py

5.2. Grounding Check

Before calling the LLM, `tutor_engine.py` first strips source-header lines from the context via `_extract_clean_context()`, then computes term overlap between the query and the cleaned context. If the score falls below 0.18, the system returns a refusal message. Note that an extractive answer is always attempted first; the grounding check only runs if the extractive step returns nothing.

```

1 def _grounding_strength(self, question, context):
2     clean_context = self._extract_clean_context(context)
3     if not clean_context:
4         return 0.0
5     q_terms = {t for t in re.findall(r"\w+", question.lower()) if
        len(t) > 2}
6     c_terms = {t for t in re.findall(r"\w+", clean_context.lower())
        if len(t) > 2}
7     if not q_terms or not c_terms:
8         return 0.0
9     return len(q_terms.intersection(c_terms)) / max(1, len(q_terms)
        )

```

Listing 2: Grounding strength check in tutor_engine.py

5.3. Supported File Formats

- `.txt`, `.md`: direct UTF-8 text read.
- `.pdf`: page text extraction via PyPDF2.

- `.csv`: rows flattened to pipe-separated text lines.
- `.json`: nested objects recursively flattened to `key: value` lines.

6. Evaluation Framework

6.1. Golden Dataset

A JSONL file (`golden_dataset.jsonl`) contains evaluation samples, each with a **question**, a reference **answer**, and the expected **source** filename.

6.2. Metrics

Metric	Description
Hit@k	Whether the expected source file appears in the top-k retrieved chunks.
MRR	Mean Reciprocal Rank: measures how highly the expected source is ranked.
Token F1	Lexical overlap (precision/recall) between generated and reference answers.
Semantic Similarity	Cosine similarity between embeddings of generated and reference answers.

These metrics together assess both *retrieval quality* (Hit@k, MRR) and *generation quality* (Token F1, Semantic Similarity), providing a complete picture of system performance.

Sample Evaluation Output

Samples: 10 Hit@1: 0.80 Hit@3: 0.90 MRR: 0.843
 Answer F1 (avg): 0.61 Semantic Similarity (avg): 0.74

7. Discussion: Mimicking vs. Understanding

A key question raised in the course is how far chatbot techniques can go, and what distinguishes mimicking from actual understanding.

This system retrieves relevant text chunks and selects sentences with high keyword overlap. It does not parse meaning, draw inferences, or build a conceptual model of the material. In that sense it is firmly on the *mimicking* side: it pattern-matches queries to stored text.

However, the grounding mechanism adds a layer of epistemic caution: the system refuses to answer when evidence is weak, which is closer to how a careful human would behave under uncertainty. The LLM component adds fluency and some degree of paraphrase, but the underlying knowledge still comes entirely from retrieved text.

This illustrates a practical boundary: RAG systems can produce useful, grounded responses without understanding, but they cannot reason beyond what is explicitly present in the retrieved context.

8. Known Limitations

- Scanned PDFs without a text layer return empty content; OCR is not supported.
- Semantic evaluation is embedding-based only; an LLM-judge could provide deeper quality assessment.
- Several LangChain APIs used are deprecated and should be upgraded.
- DOCX and other formats are not yet supported.
- The `generate_quiz()` function has no `max_tokens` limit, risking unbounded API cost.
- Uploaded filenames are not sanitized before use in file paths, creating a potential path traversal vulnerability.

9. Conclusion

This project successfully implements a RAG-based educational assistant that extends the course concept of conversational AI by adding hybrid retrieval reranking, grounding checks, persistent memory, and a quantitative evaluation framework. All main objectives were met: the system provides grounded answers, persists knowledge across restarts, allows file-level memory management, and measures its own performance.

The discussion of mimicking vs. understanding reveals that the system is effective precisely because it avoids hallucination through grounding, not because it understands the material. This is both its strength and its fundamental limitation.

Future work includes OCR support for scanned PDFs, DOCX ingestion, LLM-judge evaluation, and upgrading vulnerable dependencies.

Appendix: Key Implementation Components

Document Ingestion

```

1 def process_and_store_file(self, file_path):
2     filename = os.path.basename(file_path)
3     raw_text = self._load_file_text(file_path)
4     if not raw_text:
5         return 0
6     base_doc = Document(page_content=raw_text,
7                         metadata={"source": filename, "page": 1})
8     chunks = self.text_splitter.split_documents([base_doc])
9     ids = []
10    for idx, chunk in enumerate(chunks):
11        chunk.metadata["chunk_id"] = idx
12        stable = re.sub(r"[^a-zA-Z0-9_.-]", "_", filename.lower())
13        ids.append(f"{stable}::{chunk}::{idx}")
14    self.vector_store.add_documents(chunks, ids=ids)
15    return len(chunks)

```

Listing 3: Chunking and storing documents in database.py

Evaluation Pipeline

```

1 def _token_f1(prediction, reference):
2     pred_tokens = _normalize(prediction)
3     ref_tokens = _normalize(reference)
4     common = sum(min(pred_tokens.count(t), ref_tokens.count(t))
5                 for t in set(pred_tokens))
6     if common == 0:
7         return 0.0
8     precision = common / len(pred_tokens)
9     recall = common / len(ref_tokens)
10    return 2 * precision * recall / (precision + recall)
11
12 def _semantic_similarity(prediction, reference):
13     embedder = _get_semantic_embedder()
14     vectors = embedder.embed_documents([prediction, reference])
15     return _cosine_similarity(vectors[0], vectors[1])

```

Listing 4: Token F1 and semantic similarity in evaluation.py

Response Generation

```

1 def get_response(self, user_input, record_memory=True):
2     if self._is_chitchat(user_input):
3         return self._chitchat_answer(user_input)
4
5     context = self._retrieve_context(user_input)
6     grounding = self._grounding_strength(user_input, context)
7

```

```
8      # Extractive answer is attempted first
9      if context.strip():
10         extractive = self._extractive_answer(user_input, context)
11         if extractive:
12             return extractive
13
14      # Grounding check before calling the LLM
15      if context.strip() and grounding < 0.18:
16         return ("I don't have enough evidence in the retrieved
17                context "
18                "to answer confidently.")
19
20      response = self.client.chat.completions.create(
21         model=config.OPENAI_MODEL,
22         temperature=config.OPENAI_TEMPERATURE,
23         max_tokens=config.OPENAI_MAX_OUTPUT_TOKENS,
24         messages=[
25             {"role": "system", "content": "You are a precise
26             educational tutor."},
27             {"role": "user", "content": self._build_prompt(
28                 user_input, context)},
29         ],
30     )
31     return self._trim_to_two_sentences(
32         response.choices[0].message.content.strip())
```

Listing 5: Grounded response logic in tutor_engine.py

Appendix: Full Implementation Components

Document Ingestion

```
1 import os
2 import streamlit as st
3
4 import config
5 from audio_processor import AudioProcessor
6 from database import DocumentProcessor
7 from evaluation import evaluate_dataset, load_golden_dataset
8 from tutor_engine import TutorEngine
```

Listing 6: app.py

Configuration

```
1 import os
2 from dotenv import load_dotenv
3
4 load_dotenv()
5
6 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
7
8 TEMP_UPLOAD_DIR = os.path.join(BASE_DIR, "temp_uploads")
9 CHROMA_DB_DIR = os.path.join(BASE_DIR, "chroma_db")
10 DB_DIR = CHROMA_DB_DIR
11
12 COLLECTION_NAME = "course_documents"
13 EMBEDDING_MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
14
15 CHUNK_SIZE = 450
16 CHUNK_OVERLAP = 80
17 DEFAULT_RETRIEVAL_K = 4
18 RETRIEVAL_FETCH_K_FACTOR = 4
19
20 OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
21 OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
22 OPENAI_TEMPERATURE = float(os.getenv("OPENAI_TEMPERATURE", "0.1"))
23 OPENAI_MAX_OUTPUT_TOKENS = int(os.getenv("OPENAI_MAX_OUTPUT_TOKENS", "220"))
```

Listing 7: config.py

Database Module

```
1 import os
2 import re
3 import csv
4 import json
5 from typing import Dict, List
```

```

6 from PyPDF2 import PdfReader
7 from langchain_text_splitters import RecursiveCharacterTextSplitter
8 from langchain_community.embeddings import HuggingFaceEmbeddings
9 from langchain_community.vectorstores import Chroma
10 from langchain_core.documents import Document
11
12 import config
13
14 class DocumentProcessor:
15     def __init__(self):
16         # Setup Embeddings
17         self.embeddings = HuggingFaceEmbeddings(
18             model_name=config.EMBEDDING_MODEL_NAME,
19             model_kwargs={"device": "cpu"}
20         )
21
22         # Setup Persistent Vector Database (Chroma)
23         self.vector_store = Chroma(
24             collection_name="ai_tutor_knowledge_base",
25             embedding_function=self.embeddings,
26             persist_directory=config.DB_DIR
27         )
28
29         # Setup Text Splitter
30         self.text_splitter = RecursiveCharacterTextSplitter(
31             chunk_size=config.CHUNK_SIZE,
32             chunk_overlap=config.CHUNK_OVERLAP,
33             length_function=len,
34         )
35
36     def _read_txt_like(self, file_path: str) -> str:
37         with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
38             return f.read().strip()
39
40     def _read_pdf(self, file_path: str) -> str:
41         text_parts: List[str] = []
42         try:
43             reader = PdfReader(file_path)
44             for page in reader.pages:
45                 page_text = (page.extract_text() or "").strip()
46                 if page_text:
47                     text_parts.append(page_text)
48         except Exception:
49             return ""
50         return "\n\n".join(text_parts).strip()
51
52     def _read_csv(self, file_path: str) -> str:
53         lines: List[str] = []
54         try:

```

```

55         with open(file_path, "r", encoding="utf-8", errors="
56             ignore", newline="") as f:
57             reader = csv.reader(f)
58             for row in reader:
59                 clean_row = [c.strip() for c in row if c and c.
60                     strip()]
61                 if clean_row:
62                     lines.append(" | ".join(clean_row))
63     except Exception:
64         return ""
65     return "\n".join(lines).strip()
66
67 def _flatten_json(self, obj, prefix: str = "") -> List[str]:
68     lines: List[str] = []
69     if isinstance(obj, dict):
70         for k, v in obj.items():
71             key = f"{prefix}.{k}" if prefix else str(k)
72             lines.extend(self._flatten_json(v, key))
73     elif isinstance(obj, list):
74         for i, item in enumerate(obj):
75             key = f"{prefix}[{i}]"
76             lines.extend(self._flatten_json(item, key))
77     else:
78         value = str(obj).strip()
79         if value:
80             lines.append(f"{prefix}: {value}" if prefix else
81                 value)
82     return lines
83
84 def _read_json(self, file_path: str) -> str:
85     try:
86         with open(file_path, "r", encoding="utf-8", errors="
87             ignore") as f:
88             data = json.load(f)
89     except Exception:
90         return ""
91     return "\n".join(self._flatten_json(data)).strip()
92
93 def _load_file_text(self, file_path: str) -> str:
94     ext = os.path.splitext(file_path)[1].lower()
95     if ext in {".txt", ".md"}:
96         return self._read_txt_like(file_path)
97     if ext == ".pdf":
98         return self._read_pdf(file_path)
99     if ext == ".csv":
100         return self._read_csv(file_path)
101     if ext == ".json":
102         return self._read_json(file_path)
103     return ""
104
105 def process_and_store_file(self, file_path: str) -> int:

```

```

102     filename = os.path.basename(file_path)
103     raw_text = self._load_file_text(file_path)
104
105     if not raw_text:
106         return 0
107
108     base_doc = Document(
109         page_content=raw_text,
110         metadata={"source": filename, "page": 1}
111     )
112
113     chunks: List[Document] = self.text_splitter.split_documents
114         ([base_doc])
115     ids: List[str] = []
116     for idx, chunk in enumerate(chunks):
117         chunk.metadata["chunk_id"] = idx
118         stable_source = re.sub(r"^[a-zA-Z0-9_.-]", "_",
119             filename.lower())
120         ids.append(f"{stable_source}::chunk::{idx}")
121
122     self.vector_store.add_documents(chunks, ids=ids)
123     return len(chunks)
124
125 def _tokenize(self, text: str) -> set:
126     return {t for t in re.findall(r"\w+", text.lower()) if len(
127         t) > 2}
128
129 def _hybrid_rerank(self, query: str, docs: List[Document],
130     top_k: int) -> List[Document]:
131     if not docs:
132         return []
133
134     q_tokens = self._tokenize(query)
135     scored = []
136     for rank, doc in enumerate(docs, start=1):
137         d_tokens = self._tokenize(doc.page_content)
138         lexical = 0.0
139         if q_tokens and d_tokens:
140             lexical = len(q_tokens.intersection(d_tokens)) /
141                 max(1, len(q_tokens))
142
143         semantic = 1.0 / rank
144         score = 0.7 * semantic + 0.3 * lexical
145         scored.append((score, doc))
146
147     scored.sort(key=lambda x: x[0], reverse=True)
148
149     unique_docs: List[Document] = []
150     seen = set()
151     for _, doc in scored:
152         source = doc.metadata.get("source", "")

```

```

148         chunk_id = doc.metadata.get("chunk_id", "")
149         key = (source, chunk_id)
150         if key in seen:
151             continue
152         seen.add(key)
153         unique_docs.append(doc)
154         if len(unique_docs) >= top_k:
155             break
156     return unique_docs
157
158 def get_retriever(self, search_k: int = None):
159     if search_k is None:
160         search_k = config.DEFAULT_RETRIEVAL_K
161     fetch_k = max(8, search_k * config.RETRIEVAL_FETCH_K_FACTOR)
162
163     return self.vector_store.as_retriever(
164         search_type="mmr",
165         search_kwargs={"k": search_k, "fetch_k": fetch_k}
166     )
167
168 def retrieve_docs(self, query: str, k: int = None) -> List[
169     Document]:
170     if k is None:
171         k = config.DEFAULT_RETRIEVAL_K
172     fetch_k = max(8, k * config.RETRIEVAL_FETCH_K_FACTOR)
173     retriever = self.vector_store.as_retriever(
174         search_type="mmr",
175         search_kwargs={"k": fetch_k, "fetch_k": fetch_k}
176     )
177     try:
178         docs = retriever.get_relevant_documents(query)
179         return self._hybrid_rerank(query, docs, top_k=k)
180     except Exception:
181         return []
182
183 def list_saved_files(self) -> List[Dict[str, int]]:
184     try:
185         payload = self.vector_store.get(include=["metadatas"])
186     except Exception:
187         return []
188
189     metadatas = payload.get("metadatas", []) if isinstance(
190         payload, dict) else []
191     counts: Dict[str, int] = {}
192     for md in metadatas:
193         if not isinstance(md, dict):
194             continue
195         source = str(md.get("source", "")).strip()
196         if not source:
197             continue
198         counts[source] = counts.get(source, 0) + 1

```

```

196         rows = [{"source": k, "chunk_count": v} for k, v in counts.
197                 items()]
198         rows.sort(key=lambda x: x["source"].lower())
199         return rows
200
201     def delete_source(self, source: str) -> int:
202         ids = self._ids_for_source(source)
203         if not ids:
204             return 0
205         try:
206             self.vector_store.delete(ids=ids)
207             return len(ids)
208         except Exception:
209             return 0
210
211     def clear_all_documents(self) -> int:
212         try:
213             payload = self.vector_store.get(include=[])
214         except Exception:
215             return 0
216
217         if not isinstance(payload, dict):
218             return 0
219         ids = payload.get("ids", []) or []
220         if not ids:
221             return 0
222         try:
223             self.vector_store.delete(ids=ids)
224             return len(ids)
225         except Exception:
226             return 0

```

Listing 8: database.py

Evaluation Module

```

1 import json
2 import os
3 import re
4 from typing import Dict, List, Tuple
5 from math import sqrt
6
7 from langchain_community.embeddings import HuggingFaceEmbeddings
8
9 import config
10
11 _SEMANTIC_EMBEDDER = None
12
13 def load_golden_dataset(path: str) -> List[Dict[str, str]]:
14     if not os.path.exists(path):

```



```

15     return []
16
17     rows: List[Dict[str, str]] = []
18     with open(path, "r", encoding="utf-8") as f:
19         for line in f:
20             line = line.strip()
21             if not line:
22                 continue
23             try:
24                 item = json.loads(line)
25             except json.JSONDecodeError:
26                 continue
27
28             question = str(item.get("question", "")).strip()
29             answer = str(item.get("answer", "")).strip()
30             source = str(item.get("source", "")).strip()
31             if question and answer:
32                 rows.append({
33                     "question": question,
34                     "answer": answer,
35                     "source": source,
36                 })
37     return rows
38
39 def _normalize(text: str) -> List[str]:
40     text = text.lower()
41     text = re.sub(r"[^\w\s]", " ", text)
42     return [t for t in text.split() if t]
43
44 def _token_f1(prediction: str, reference: str) -> float:
45     pred_tokens = _normalize(prediction)
46     ref_tokens = _normalize(reference)
47     if not pred_tokens or not ref_tokens:
48         return 0.0
49
50     pred_counts: Dict[str, int] = {}
51     ref_counts: Dict[str, int] = {}
52     for t in pred_tokens:
53         pred_counts[t] = pred_counts.get(t, 0) + 1
54     for t in ref_tokens:
55         ref_counts[t] = ref_counts.get(t, 0) + 1
56
57     common = 0
58     for token, p_count in pred_counts.items():
59         if token in ref_counts:
60             common += min(p_count, ref_counts[token])
61
62     if common == 0:
63         return 0.0
64
65     precision = common / len(pred_tokens)

```

```

66     recall = common / len(ref_tokens)
67     return 2 * precision * recall / (precision + recall)
68
69 def _get_semantic_embedder():
70     global _SEMANTIC_EMBEDDER
71     if _SEMANTIC_EMBEDDER is None:
72         _SEMANTIC_EMBEDDER = HuggingFaceEmbeddings(
73             model_name=config.EMBEDDING_MODEL_NAME,
74             model_kwargs={"device": "cpu"},
75         )
76     return _SEMANTIC_EMBEDDER
77
78 def _cosine_similarity(a: List[float], b: List[float]) -> float:
79     if not a or not b or len(a) != len(b):
80         return 0.0
81     dot = sum(x * y for x, y in zip(a, b))
82     na = sqrt(sum(x * x for x in a))
83     nb = sqrt(sum(y * y for y in b))
84     if na == 0.0 or nb == 0.0:
85         return 0.0
86     return dot / (na * nb)
87
88 def _semantic_similarity(prediction: str, reference: str) -> float:
89     if not prediction.strip() or not reference.strip():
90         return 0.0
91     try:
92         embedder = _get_semantic_embedder()
93         vectors = embedder.embed_documents([prediction, reference])
94         return _cosine_similarity(vectors[0], vectors[1])
95     except Exception:
96         return 0.0
97
98 def evaluate_dataset(doc_processor, tutor_engine, dataset: List[
99     Dict[str, str]], k_values: Tuple[int, ...] = (1, 3, 5)) -> Dict:
100     if not dataset:
101         return {
102             "num_samples": 0,
103             "hit_rates": {"Hit@{k}": 0.0 for k in k_values},
104             "mrr": 0.0,
105             "avg_f1": 0.0,
106             "avg_semantic_similarity": 0.0,
107             "rows": [],
108         }
109
110     hit_counts = {k: 0 for k in k_values}
111     mrr_total = 0.0
112     f1_total = 0.0
113     semantic_total = 0.0
114     rows = []
115
116     max_k = max(k_values)

```

```

116
117     for sample in dataset:
118         question = sample["question"]
119         expected_answer = sample["answer"]
120         expected_source = sample.get("source", "")
121
122         docs = doc_processor.retrieve_docs(question, k=max_k)
123         retrieved_sources = [str(d.metadata.get("source", "")) for
124                               d in docs]
125
126         rank = None
127         if expected_source:
128             for idx, src in enumerate(retrieved_sources, start=1):
129                 if src == expected_source:
130                     rank = idx
131                     break
132
133             for k in k_values:
134                 if rank is not None and rank <= k:
135                     hit_counts[k] += 1
136             if rank is not None:
137                 mrr_total += 1.0 / rank
138
139         predicted_answer = tutor_engine.get_response(question,
140                                                       record_memory=False)
141         f1_score = _token_f1(predicted_answer, expected_answer)
142         semantic_sim = _semantic_similarity(predicted_answer,
143                                             expected_answer)
144         f1_total += f1_score
145         semantic_total += semantic_sim
146
147         rows.append({
148             "question": question,
149             "expected_source": expected_source,
150             "retrieved_top1": retrieved_sources[0] if
151                 retrieved_sources else "",
152             "rank_of_expected_source": rank if rank is not None
153                 else "not_found",
154             "answer_f1": round(f1_score, 3),
155             "semantic_similarity": round(semantic_sim, 3),
156         })
157
158     n = len(dataset)
159     hit_rates = {f"Hit@{k}": round(hit_counts[k] / n, 3) for k in
160                  k_values}
161     mrr = round(mrr_total / n, 3)
162     avg_f1 = round(f1_total / n, 3)
163     avg_semantic_similarity = round(semantic_total / n, 3)
164
165     return {
166         "num_samples": n,

```

```

161         "hit_rates": hit_rates,
162         "mrr": mrr,
163         "avg_f1": avg_f1,
164         "avg_semantic_similarity": avg_semantic_similarity,
165         "rows": rows,
166     }
167
168 \subsection*{Main Application}
169 \begin{lstlisting}[caption={app.py (continued)}]
170
171
172 if "doc_processor" not in st.session_state:
173     st.session_state.doc_processor = DocumentProcessor()
174 if "tutor_engine" not in st.session_state:
175     st.session_state.tutor_engine = TutorEngine(
176         retriever=st.session_state.doc_processor.get_retriever(
177             search_k=config.DEFAULT_RETRIEVAL_K
178         )
179     )
180 if "audio_processor" not in st.session_state:
181     st.session_state.audio_processor = AudioProcessor()
182 if "messages" not in st.session_state:
183     st.session_state.messages = [
184         {
185             "role": "assistant",
186             "content": "Hello! I am your Socratic AI Tutor. Ask me
187                         any question about your uploaded study files.",
188         }
189     ]
190 if "retrieval_k" not in st.session_state:
191     st.session_state.retrieval_k = config.DEFAULT_RETRIEVAL_K
192
193 st.set_page_config(page_title="Socratic AI Tutor", page_icon="
194     graduation", layout="wide")
195 st.title("Socratic AI Tutor with Persistent Memory")
196
197 def render_evaluation_results():
198     eval_result = st.session_state.get("eval_result")
199     if not eval_result:
200         st.info("No evaluation result yet. Click 'Evaluate Model'
201                 first.")
202         return
203
204     if eval_result["num_samples"] == 0:
205         st.info(
206             "No golden dataset found. Create 'golden_dataset.jsonl'
207             in the project root to run evaluation."
208         )
209     else:
210         c1, c2, c3, c4 = st.columns(4)
211         c1.metric("Samples", eval_result["num_samples"])

```

```

208     c2.metric("Hit@1", eval_result["hit_rates"]["Hit@1"])
209     c3.metric("Hit@3", eval_result["hit_rates"]["Hit@3"])
210     c4.metric("MRR", eval_result["mrr"])
211     st.metric("Answer F1 (avg)", eval_result["avg_f1"])
212     st.metric("Semantic Similarity (avg)", eval_result.get("
213         avg_semantic_similarity", 0.0))
214     st.dataframe(eval_result["rows"], use_container_width=True,
215                 height=360)
216
217 def render_saved_files_panel():
218     st.subheader("Saved Files")
219     saved_files = st.session_state.doc_processor.list_saved_files()
220     if not saved_files:
221         st.caption("No files saved yet.")
222         return
223     for row in saved_files:
224         c1, c2 = st.columns([0.88, 0.12], gap="small")
225         c1.markdown(f"‘{row['source']}' ‘({row['chunk_count']}
226             chunks)")
227         if c2.button("X", key=f"delete_{row['source']}", help="
228             Delete this file", use_container_width=True):
229             deleted = st.session_state.doc_processor.delete_source(
230                 row["source"])
231             st.session_state.tutor_engine.update_retriever(
232                 st.session_state.doc_processor.get_retriever(
233                     search_k=st.session_state.retrieval_k
234                 )
235             )
236             if deleted > 0:
237                 st.success(f"Deleted ‘{row['source']}' ‘({deleted}
238                     chunks)")
239             else:
240                 st.warning(f"Could not delete ‘{row['source']}'“)
241                 st.rerun()
242
243 left_col, center_col, right_col = st.columns([1.0, 1.8, 1.0], gap="
244     large")
245
246 with left_col:
247     st.header("Knowledge Base")
248     st.write("Upload study files, save them, and see what is
249         currently stored.")
250
251     retrieval_k = st.slider(
252         "Retrieval Top-k",
253         min_value=1,
254         max_value=8,
255         value=st.session_state.retrieval_k,
256         help="Lower values are stricter; higher values provide
257             broader context.",
258         key="left_retrieval_k",

```

```

250 )
251 if retrieval_k != st.session_state.retrieval_k:
252     st.session_state.retrieval_k = retrieval_k
253     st.session_state.tutor_engine.update_retriever(
254         st.session_state.doc_processor.get_retriever(
255             search_k=st.session_state.retrieval_k
256         )
257     )
258
259 uploaded_file = st.file_uploader(
260     "Upload a file",
261     type=["txt", "md", "pdf", "csv", "json"],
262     key="left_file_uploader",
263 )
264 if st.button("Process & Save Document", key="left_save_btn")
265 and uploaded_file:
266     with st.spinner("Processing document..."):
267         temp_path = os.path.join(config.TEMP_UPLOAD_DIR,
268             uploaded_file.name)
269         with open(temp_path, "wb") as f:
270             f.write(uploaded_file.getbuffer())
271
272         num_chunks = st.session_state.doc_processor.
273             process_and_store_file(temp_path)
274         st.session_state.tutor_engine.update_retriever(
275             st.session_state.doc_processor.get_retriever(
276                 search_k=st.session_state.retrieval_k
277             )
278         )
279
280         if num_chunks == 0:
281             st.warning("File was empty or unsupported. Nothing
282                 was added.")
283         else:
284             st.success(f"Saved: '{uploaded_file.name}' ({
285                 num_chunks} chunks)")
286             st.rerun()
287
288 st.divider()
289 if st.button("Reset Knowledge Base (Delete All)", key="
290 left_reset_all_btn", type="secondary"):
291     deleted_total = st.session_state.doc_processor.
292         clear_all_documents()
293     st.session_state.tutor_engine.update_retriever(
294         st.session_state.doc_processor.get_retriever(
295             search_k=st.session_state.retrieval_k
296         )
297     )
298
299 if deleted_total > 0:
300     st.success(f"All saved knowledge deleted ({
301         deleted_total} chunks).")

```

```

293         else:
294             st.info("Knowledge base was already empty.")
295             st.rerun()
296
297     render_saved_files_panel()
298
299     with center_col:
300         st.header("Chat")
301         for msg in st.session_state.messages:
302             with st.chat_message(msg["role"]):
303                 st.write(msg["content"])
304                 if msg["role"] == "assistant" and msg.get("sources"):
305                     source_lines = []
306                     for s in msg["sources"]:
307                         source_lines.append(
308                             f"- source: '{s.get('source', 'unknown')}',
309                               page: '{s.get('page', '?')}', chunk: '{s.get('chunk_id', '?')}'"
310                         )
311                     st.markdown("Retrieved evidence:\n" + "\n".join(
312                         source_lines))
313
314         st.divider()
315         with st.form("center_chat_form", clear_on_submit=True):
316             input_col, send_col = st.columns([0.86, 0.14], gap="small")
317             user_input = input_col.text_input(
318                 "Message",
319                 key="center_chat_input",
320                 placeholder="Ask a question...",
321                 label_visibility="collapsed",
322             )
323             send_clicked = send_col.form_submit_button("Send",
324                                                         use_container_width=True)
325
326             if send_clicked and user_input.strip():
327                 with st.spinner("Thinking..."):
328                     st.session_state.messages.append({"role": "user", "content": user_input.strip()})
329                     response_text = st.session_state.tutor_engine.get_response(user_input.strip())
330                     st.session_state.messages.append(
331                         {
332                             "role": "assistant",
333                             "content": response_text,
334                             "sources": st.session_state.tutor_engine.get_last_retrieved_sources(),
335                         }
336                     )
337                 st.rerun()
338
339     with right_col:

```

```

337     st.header("Assessment")
338     has_saved_files = len(st.session_state.doc_processor.
339         list_saved_files()) > 0
340     st.button(
341         "Generate Contextual Quiz",
342         disabled=True,
343         help="Coming soon (outside MVP scope).",
344         key="right_quiz_btn",
345     )
346     dataset_path = os.path.join(config.BASE_DIR, "golden_dataset.
347         jsonl")
348     if st.button(
349         "Evaluate Model",
350         key="right_eval_btn",
351         disabled=not has_saved_files,
352         help="Save at least one file first." if not has_saved_files
353         else None,
354     ):
355         with st.spinner("Running retrieval/generation evaluation...
356             "):
357             dataset = load_golden_dataset(dataset_path)
358             result = evaluate_dataset(
359                 doc_processor=st.session_state.doc_processor,
360                 tutor_engine=st.session_state.tutor_engine,
361                 dataset=dataset,
362                 k_values=(1, 3, 5),
363             )
364             st.session_state.eval_result = result
365     with st.expander("View Evaluation Results", expanded=False):
366         render_evaluation_results()

```

Listing 9: evaluation.py

Additional Files

```

1 streamlit==1.36.0
2 langchain==0.2.6
3 langchain-community==0.2.6
4 langchain-core==0.2.11
5 chromadb==0.5.3
6 sentence-transformers==2.7.0
7 pypdf2==3.0.1
8 python-dotenv==1.0.1
9 gtts==2.5.1
10 audio-recorder-streamlit==0.0.10
11 openai==1.30.5
12 httpx==0.27.2

```

Listing 10: requirements.txt


```

1 import requests
2 import json
3
4 urls = [
5     "http://localhost:11434/api/tags",
6     "https://mitsubishi-refers-raid-rail.trycloudflare.com/api/tags"
7 ]
8
9 for url in urls:
10     print(f"Checking {url}...")
11     try:
12         response = requests.get(url, timeout=5)
13         print(f"Status Code: {response.status_code}")
14         if response.status_code == 200:
15             print(f"Ollama is reachable! {response.json()}")
16         else:
17             print(f"Not reachable. Response: {response.text[:100]}")
18     except Exception as e:
19         print(f"Error: {e}")

```

Listing 11: test_connection.py

```

1 {
2     "$schema": "https://schema.management.azure.com/schemas
3     /2019-04-01/deploymentTemplate.json#",
4     "contentVersion": "1.0.0.0",
5     "parameters": {
6         "vmName": {
7             "type": "string",
8             "defaultValue": "ai-tutor-vm"
9         },
10        "adminUsername": {
11            "type": "string",
12            "defaultValue": "azureuser"
13        },
14        "adminPassword": {
15            "type": "secureString"
16        },
17        "vmSize": {
18            "type": "string",
19            "defaultValue": "Standard_B2ms"
20        },
21        "location": {
22            "type": "string",
23            "defaultValue": "[resourceGroup().location]"
24        },
25        "variables": {
26            "vnetName": "[concat(parameters('vmName'), '-vnet')]",

```

```

27     "nsgName": "[concat(parameters('vmName'), '-nsg')]",
28     "publicIpName": "[concat(parameters('vmName'), '-pip')]"
29 },
30 "resources": [
31     {
32         "type": "Microsoft.Network/networkSecurityGroups",
33         "apiVersion": "2023-09-01",
34         "name": "[variables('nsgName')]",
35         "location": "[parameters('location')]",
36         "properties": {
37             "securityRules": [
38                 {
39                     "name": "Allow-SSH",
40                     "properties": {
41                         "priority": 1000,
42                         "access": "Allow",
43                         "direction": "Inbound",
44                         "protocol": "Tcp",
45                         "destinationPortRange": "22"
46                     }
47                 },
48                 {
49                     "name": "Allow-Streamlit",
50                     "properties": {
51                         "priority": 1010,
52                         "access": "Allow",
53                         "direction": "Inbound",
54                         "protocol": "Tcp",
55                         "destinationPortRange": "8501"
56                     }
57                 }
58             ]
59         }
60     }
61 ]
62 }

```

Listing 12: azure-deploy.json (ARM Template)

```

1 {
2     "$schema": "https://schema.management.azure.com/schemas
3     /2019-04-01/deploymentParameters.json#",
4     "contentVersion": "1.0.0.0",
5     "parameters": {
6         "vmName": {
7             "value": "ai-tutor-vm"
8         },
9         "adminUsername": {
10            "value": "azureuser"
11        },
12        "adminPassword": {
13            "value": "YourStrongPassword_123!"
14        }
15    }
16 }

```

```

13     },
14     "vmSize": {
15         "value": "Standard_B2ms"
16     },
17     "location": {
18         "value": "westeurope"
19     }
20 }
21 }

```

Listing 13: azure-deploy.parameters.json

```

1 {"question":"What is the main topic of chapter 1?","answer":"The
   main topic of chapter 1 is ...","source":"your_file_name.txt"}
2 {"question":"Define term X based on the text.","answer":"Term X
   means ...","source":"your_file_name.txt"}
3 {"question":"What are the two key differences between A and B?","
   answer":"The two key differences are ...","source":"
   your_file_name.txt"}

```

Listing 14: golden_dataset.jsonl (sample)

Tutor Engine Module

```

1 import re
2
3 from openai import OpenAI, RateLimitError, AuthenticationError,
   APIConnectionError, OpenAIError
4 from langchain.memory import ConversationBufferMemory
5
6 import config
7
8 class TutorEngine:
9     def __init__(self, retriever):
10         self.client = OpenAI(api_key=config.OPENAI_API_KEY)
11         self.retriever = retriever
12         self.last_retrieved_sources = []
13         self.memory = ConversationBufferMemory(
14             memory_key="chat_history",
15             return_messages=True
16         )
17
18     def _retrieve_context(self, question: str) -> str:
19         try:
20             docs = self.retriever.get_relevant_documents(question)
21         except Exception:
22             docs = []
23         self.last_retrieved_sources = []
24         if not docs:
25             return ""
26         context_parts = []

```

```

27     seen = set()
28     for doc in docs:
29         source = doc.metadata.get("source", "unknown")
30         page = doc.metadata.get("page", "?")
31         chunk_id = doc.metadata.get("chunk_id", "?")
32         key = (source, page, chunk_id)
33         if key in seen:
34             continue
35         seen.add(key)
36         self.last_retrieved_sources.append(
37             {"source": source, "page": page, "chunk_id":
38              chunk_id}
39         )
40         context_parts.append(
41             f"[Source: {source}, page {page}, chunk {chunk_id}
42              ]\n{doc.page_content}"
43         )
44     return "\n\n".join(context_parts)
45
46 def _extractive_answer(self, question: str, context: str) ->
47     str:
48     clean_context = self._extract_clean_context(context)
49     if not clean_context:
50         return ""
51     question_terms = {
52         t for t in re.findall(r"\w+", question.lower())
53         if len(t) > 2
54     }
55     stop_terms = {
56         "what", "when", "where", "which", "does", "about", "
57         from", "with",
58         "that", "this", "have", "your", "give", "common", "used
59         ", "mean",
60         "important", "why", "is", "the", "are", "for", "and"
61     }
62     question_terms = {t for t in question_terms if t not in
63                       stop_terms}
64     if not question_terms:
65         return ""
66     sentences = re.split(r"(?<=[.!?])\s+", clean_context)
67     scored = []
68     for sent in sentences:
69         sent = sent.strip()
70         if not sent:
71             continue
72         sent_terms = {t for t in re.findall(r"\w+", sent.lower
73         ()) if len(t) > 2}
74         if not sent_terms:
75             continue
76         overlap = len(question_terms.intersection(sent_terms))
77         if overlap == 0:

```

```

71         continue
72         score = overlap / max(1, len(question_terms))
73         scored.append((score, sent))
74     if not scored:
75         return ""
76     scored.sort(key=lambda x: x[0], reverse=True)
77     best_score, best_sentence = scored[0]
78     if best_score < 0.20:
79         return ""
80     return self._trim_to_two_sentences(best_sentence)
81
82 def _grounding_strength(self, question: str, context: str) ->
float:
83     clean_context = self._extract_clean_context(context)
84     if not clean_context:
85         return 0.0
86     q_terms = {t for t in re.findall(r"\w+", question.lower())
87                 if len(t) > 2}
88     c_terms = {t for t in re.findall(r"\w+", clean_context.
89                                     lower()) if len(t) > 2}
90     if not q_terms or not c_terms:
91         return 0.0
92     overlap = len(q_terms.intersection(c_terms))
93     return overlap / max(1, len(q_terms))
94
95 def get_response(self, user_input: str, record_memory: bool =
True) -> str:
96     if self._is_chitchat(user_input):
97         self.last_retrieved_sources = []
98         answer = self._chitchat_answer(user_input)
99         if record_memory:
100             self.memory.chat_memory.add_user_message(user_input
101                                                         )
102             self.memory.chat_memory.add_ai_message(answer)
103         return answer
104     context = self._retrieve_context(user_input)
105     grounding = self._grounding_strength(user_input, context)
106     if context.strip():
107         extractive = self._extractive_answer(user_input,
108                                             context)
109         if extractive:
110             answer = extractive
111             if record_memory:
112                 self.memory.chat_memory.add_user_message(
113                     user_input)
114                 self.memory.chat_memory.add_ai_message(answer)
115             return answer
116     if context.strip():
117         if grounding < 0.18:
118             answer = (

```

```

114         "I don't have enough evidence in the retrieved
           context to answer confidently."
115     )
116     if record_memory:
117         self.memory.chat_memory.add_user_message(
           user_input)
118         self.memory.chat_memory.add_ai_message(answer)
119     return answer
120 try:
121     response = self.client.chat.completions.create(
122         model=config.OPENAI_MODEL,
123         temperature=config.OPENAI_TEMPERATURE,
124         max_tokens=config.OPENAI_MAX_OUTPUT_TOKENS,
125         messages=[
126             {"role": "system", "content": "You are a
           precise educational tutor."},
127             {"role": "user", "content": self._build_prompt(
           user_input, context)},
128         ],
129     )
130     answer = self._trim_to_two_sentences((response.choices
           [0].message.content or "").strip())
131 except OpenAIError:
132     answer = self._fallback_answer(user_input, context)
133 if record_memory:
134     self.memory.chat_memory.add_user_message(user_input)
135     self.memory.chat_memory.add_ai_message(answer)
136 return answer

```

Listing 15: tutor_engine.py

Audio Processor Module

```

1 import base64
2 from gtts import gTTS
3 import os
4
5 class AudioProcessor:
6     def __init__(self):
7         self.temp_audio_file = "temp_response.mp3"
8
9     def text_to_speech_html(self, text: str, lang: str = 'en') ->
       str:
10         # Create TTS
11         tts = gTTS(text=text, lang=lang, slow=False)
12         tts.save(self.temp_audio_file)
13
14         # Read the audio file
15         with open(self.temp_audio_file, "rb") as f:
16             data = f.read()
17             b64 = base64.b64encode(data).decode()

```

```

18         # Create an invisible HTML audio tag that plays
19         # automatically
20         md = f"""
21             <audio autoplay="true" style="display:none;">
22             <source src="data:audio/mp3;base64,{b64}" type="audio/
23             mp3">
24             </audio>
25             """
26         # Cleanup temp file
27         if os.path.exists(self.temp_audio_file):
28             os.remove(self.temp_audio_file)
29
30         return md

```

Listing 16: audio_processor.py

Docker Configuration

```

1 services:
2     aitutor:
3         build: .
4         container_name: socratic_ai_tutor
5         ports:
6             - "8501:8501"
7         volumes:
8             - aitutor_chroma_data:/app/chroma_db
9             - aitutor_uploads:/app/temp_uploads
10        env_file:
11            - .env
12        restart: unless-stopped
13
14 volumes:
15     aitutor_chroma_data:
16     aitutor_uploads:

```

Listing 17: docker-compose.yml

```

1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 ENV PYTHONDONTWRITEBYTECODE=1
6 ENV PYTHONUNBUFFERED=1
7 ENV STREAMLIT_SERVER_PORT=8501
8 ENV STREAMLIT_SERVER_ADDRESS=0.0.0.0
9
10 RUN apt-get update && apt-get install -y \
11     build-essential \
12     curl \

```

```
13     && rm -rf /var/lib/apt/lists/*
14
15 COPY requirements.txt .
16
17 RUN pip install --no-cache-dir --upgrade pip && \
18     pip install --no-cache-dir -r requirements.txt
19
20 COPY . .
21
22 RUN mkdir -p /app/chroma_db /app/temp_uploads
23
24 EXPOSE 8501
25
26 CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.\
    address=0.0.0.0"]
```

Listing 18: Dockerfile